



Setup Guide (Ollama + MCP)

Guida per ricreare da zero l'agente che analizza la sicurezza di una repository usando un **modello Ollama locale** che esegue **query CodeQL** esposte come **tool MCP**. Contiene tutti gli script e i check eseguiti.

1. Cosa fa

L'agente, data una repository di codice:

1. enumera i file Python;
2. costruisce un **database CodeQL**;
3. esegue la **suite di sicurezza** CodeQL;
4. ispeziona i finding nel codice sorgente;
5. genera un **report di sicurezza** in Markdown (per file, con severità e remediation).

Il modello Ollama non parla MCP nativamente: serve un **host** (loop Python) che fa da ponte tra il tool-calling del modello e il server MCP che incapsula CodeQL.

2. Architettura

```
Ollama LLM          agent.py (host)          codeql_mcp
_server.py          CodeQL CLI
(qwen2.5-coder:14b) --tool_calls--> loop --MCP/stdio--> tool:
create_db/analyze --> + query locali
```

- `codeql_mcp_server.py` — server MCP (FastMCP) che incapsula la CLI CodeQL.
Tool: `list_python_files`, `create_codeql_database`, `analyze_database`, `run_custom_query`, `read_file_snippet`.

- `agent.py` — host: avvia il server come subprocess stdio, traduce i tool MCP nello schema function-calling di Ollama, gira il loop `chat → tool → risultato` finché il modello scrive il report.
- `config.py` — modello, percorsi CodeQL, suite di default, timeout.

3. Prerequisiti (ambiente verificato)

Componente	Stato verificato
CodeQL CLI	<code>2.25.2</code> , su PATH
Query CodeQL	checkout locale <code>vscode-codeql-starter/ql</code> (usato via <code>--search-path</code> , niente download pack)
Ollama	in esecuzione, modello <code>qwen2.5-coder:14b</code>
Python	<code>3.13</code>
Pacchetti Python	<code>mcp</code> (già presente), <code>ollama</code> (installato)

La cartella del progetto: `C:\Users\ladyc\Documents\TRAINEESHIP\CodeQL_Security_Agent`.

4. Setup

```
cd C:\Users\ladyc\Documents\TRAINEESHIP\CodeQL_Security_Agent
pip install -r requirements.txt
```

`requirements.txt`:

```
mcp>=1.0.0
ollama>=0.4.0
```

5. Script

5.1 `config.py`

```
"""Central configuration. Override via environment variables
(.env)."""
```

```

import os
from pathlib import Path

# --- Ollama ---
OLLAMA_HOST = os.environ.get("OLLAMA_HOST", "http://localhost:11434")
AGENT_MODEL = os.environ.get("AGENT_MODEL", "qwen2.5-coder:14b")

# --- CodeQL ---
# `codeql` must be on PATH (verified: 2.25.2). Override with
# a full path if needed.
CODEQL_BIN = os.environ.get("CODEQL_BIN", "codeql")

# Local CodeQL query checkout (vscode-codeql-starter). Used as
# --search-path so
# the standard query packs resolve without downloading anything.
CODEQL_SEARCH_PATH = os.environ.get(
    "CODEQL_SEARCH_PATH",
    r"C:\Users\ladyc\Documents\TRAINEESHIP\vscode-codeql-starter\ql",
)

# Standard query suite run by analyze_database() when no custom
# query is given.
DEFAULT_SUITE = os.environ.get(
    "CODEQL_SUITE",
    r"C:\Users\ladyc\Documents\TRAINEESHIP\vscode-codeql-starter\ql\python\ql\src\codeql-suites\python-security-extended.qls",
)

# Where CodeQL databases and SARIF outputs are written.
WORK_DIR = Path(os.environ.get("WORK_DIR", Path(__file__).parent / "_work"))

```

```

WORK_DIR.mkdir(exist_ok=True)

# Per-CodeQL-command timeout (seconds). DB creation on large
# repos is slow.
CODEQL_TIMEOUT = int(os.environ.get("CODEQL_TIMEOUT", "180
0"))

```

5.2 `codeql_mcp_server.py`

```

"""MCP server exposing CodeQL as tools.

Tools: list_python_files, create_codeql_database, analyze_dat
abase,
run_custom_query, read_file_snippet.
"""
from __future__ import annotations

import hashlib
import json
import subprocess
from pathlib import Path

from mcp.server.fastmcp import FastMCP

import config

mcp = FastMCP("codeql")

def _run(cmd, timeout=config.CODEQL_TIMEOUT):
    proc = subprocess.run(
        cmd, capture_output=True, text=True, timeout=timeout,
        encoding="utf-8", errors="replace",
    )
    return proc.returncode, proc.stdout, proc.stderr

```

```

def _db_path_for(repo_path):
    digest = hashlib.sha1(str(Path(repo_path).resolve()).encode()).hexdigest()[:10]
    name = Path(repo_path).resolve().name
    return config.WORK_DIR / f"db_{name}_{digest}"

def _parse_sarif(sarif_path):
    data = json.loads(Path(sarif_path).read_text(encoding="utf-8"))
    findings = []
    for run in data.get("runs", []):
        rules = {}
        for rule in run.get("tool", {}).get("driver", {}).get("rules", []):
            props = rule.get("properties", {})
            rules[rule.get("id")] = {
                "severity": props.get("security-severity") or
                props.get("problem.severity", ""),
                "name": rule.get("name", rule.get("id", "")),
                "tags": props.get("tags", []),
            }
        for res in run.get("results", []):
            rid = res.get("ruleId", "")
            loc = (res.get("locations") or [{}])[0].get("physicalLocation", {})
            findings.append({
                "rule_id": rid,
                "rule_name": rules.get(rid, {}).get("name", rid),
                "level": res.get("level", "warning"),
                "security_severity": rules.get(rid, {}).get("severity", ""),
                "tags": rules.get(rid, {}).get("tags", []),
            })

```

```

        "message": res.get("message", {}).get("text",
""),
        "file": loc.get("artifactLocation", {}).get
("uri", ""),
        "line": loc.get("region", {}).get("startLin
e"),
    })
    return findings

```

```

@mcp.tool()
def list_python_files(repo_path: str, max_files: int = 500) -
> str:
    """List Python source files in a repository (skips venv/s
ite-packages/.git)."""
    root = Path(repo_path)
    if not root.is_dir():
        return json.dumps({"error": f"Not a directory: {repo_
path}"})
    skip = {".git", ".venv", "venv", "__pycache__", "site-pac
kages", "node_modules"}
    files = []
    for p in root.rglob("*.py"):
        if any(part in skip for part in p.parts):
            continue
        files.append(str(p.relative_to(root)))
        if len(files) >= max_files:
            break
    return json.dumps({"count": len(files), "files": files},
indent=2)

```

```

@mcp.tool()
def create_codeql_database(repo_path: str, language: str = "p
ython") -> str:
    """Create a CodeQL database from a repository. Returns th

```

```

e database path."""
    repo = Path(repo_path)
    if not repo.is_dir():
        return json.dumps({"error": f"Not a directory: {repo_
path}"})
    db = _db_path_for(repo_path)
    cmd = [
        config.CODEQL_BIN, "database", "create", str(db),
        f"--language={language}", f"--source-root={repo}", "-
-overwrite",
    ]
    try:
        rc, out, err = _run(cmd)
    except subprocess.TimeoutExpired:
        return json.dumps({"error": "database create timed ou
t", "db_path": str(db)})
    if rc != 0:
        return json.dumps({"error": "database create failed",
"stderr": err[-2000:]})
    return json.dumps({"db_path": str(db), "language": langua
ge, "status": "created"})

```

```

@mcp.tool()
def analyze_database(db_path: str, suite: str = "") -> str:
    """Run a CodeQL security suite against a database and ret
urn parsed findings."""
    db = Path(db_path)
    if not db.is_dir():
        return json.dumps({"error": f"DB not found: {db_pat
h}"})
    suite = suite or config.DEFAULT_SUITE
    sarif = Path(db_path).parent / f"{Path(db_path).name}.sar
if"
    cmd = [
        config.CODEQL_BIN, "database", "analyze", str(db), su

```

```

ite,
    f"--search-path={config.CODEQL_SEARCH_PATH}",
    "--format=sarifv2.1.0", f"--output={sarif}", "--reru
n",
]
try:
    rc, out, err = _run(cmd)
except subprocess.TimeoutExpired:
    return json.dumps({"error": "analyze timed out"})
if rc != 0:
    return json.dumps({"error": "analyze failed", "stderr": err[-2000:]})
findings = _parse_sarif(sarif)
return json.dumps(
    {"sarif_path": str(sarif), "finding_count": len(findings), "findings": findings},
    indent=2,
)

@mcp.tool()
def run_custom_query(db_path: str, query_path: str) -> str:
    """Run a single custom .ql query file against a database."""
    db, q = Path(db_path), Path(query_path)
    if not db.is_dir():
        return json.dumps({"error": f"DB not found: {db_path}"})
    if not q.is_file():
        return json.dumps({"error": f"Query not found: {query_path}"})
    sarif = Path(db_path).parent / f"{q.stem}.sarif"
    cmd = [
        config.CODEQL_BIN, "database", "analyze", str(db), str(q),
        f"--search-path={config.CODEQL_SEARCH_PATH}",

```

```

        "--format=sarifv2.1.0", f"--output={sarif}", "--reru
n",
    ]
    try:
        rc, out, err = _run(cmd)
    except subprocess.TimeoutExpired:
        return json.dumps({"error": "query timed out"})
    if rc != 0:
        return json.dumps({"error": "query failed", "stderr":
err[-2000:]})
    findings = _parse_sarif(sarif)
    return json.dumps({"finding_count": len(findings), "findi
ngs": findings}, indent=2)

@mcp.tool()
def read_file_snippet(file_path: str, start_line: int = 1, en
d_line: int = 40) -> str:
    """Read a slice of a source file so the model can inspect
a finding's context."""
    p = Path(file_path)
    if not p.is_file():
        return json.dumps({"error": f"File not found: {file_p
ath}"})
    lines = p.read_text(encoding="utf-8", errors="replace").s
plitlines()
    lo, hi = max(1, start_line), min(len(lines), end_line)
    numbered = [f"{i}: {lines[i - 1]}" for i in range(lo, hi
+ 1)]
    return json.dumps({"file": file_path, "lines": "\n".join
(numbered)})

if __name__ == "__main__":
    mcp.run()

```

5.3 `agent.py`

```
"""Security analysis agent: local Ollama model + CodeQL via MCP.
```

```
Usage:
```

```
    python agent.py <repo_path> [--model qwen2.5-coder:14b]
    [--report report.md]
    """
```

```
from __future__ import annotations
```

```
import argparse
```

```
import asyncio
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
from mcp import ClientSession, StdioServerParameters
```

```
from mcp.client.stdio import stdio_client
```

```
from ollama import AsyncClient
```

```
import config
```

```
SYSTEM_PROMPT = """You are a security analysis agent. You analyze a code repository \
for vulnerabilities using CodeQL, which is available to you through tools.
```

```
Follow this process:
```

1. Call `list_python_files` to see the repository contents.
2. Call `create_codeql_database` on the repository root to build the analysis DB.
3. Call `analyze_database` with the returned `db_path` to get CodeQL findings.
4. For each finding, optionally call `read_file_snippet` to inspect the vulnerable code.

5. Produce a final SECURITY REPORT in Markdown. Do NOT call any more tools once you start writing the report.

The report must include, per finding: the CWE/rule, severity, file and line, a short explanation, and a concrete remediation. Group findings by file. End with a summary table (severity counts) and an overall risk verdict. Rely on CodeQL evidence, do not invent vulnerabilities."""

```
def mcp_tools_to_ollama(tools):
    return [
        {
            "type": "function",
            "function": {
                "name": t.name,
                "description": t.description or "",
                "parameters": t.inputSchema,
            },
        }
        for t in tools
    ]

def _tool_result_text(result):
    parts = []
    for block in result.content:
        parts.append(getattr(block, "text", "") or "")
    return "\n".join(parts) or "(empty result)"

async def run_agent(repo_path, model, report_path, max_steps=
30):
```

```

server = StdioServerParameters(
    command=sys.executable,
    args=[str(Path(__file__).parent / "codeql_mcp_server.
py")],
)
client = AsyncClient(host=config.OLLAMA_HOST)

async with stdio_client(server) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()
        tools = mcp_tools_to_ollama((await session.list_t
ools()).tools)

        messages = [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"Analyze the rep
ository at: {repo_path}"},
        ]

        for step in range(max_steps):
            resp = await client.chat(model=model, message
s=messages, tools=tools)
            msg = resp["message"]
            messages.append(msg)

            calls = msg.get("tool_calls") or []
            if not calls:
                report = msg.get("content", "")
                Path(report_path).write_text(report, enco
ding="utf-8")
                return report

            for call in calls:
                fn = call["function"]
                name = fn["name"]
                args = fn["arguments"]

```

```

        if isinstance(args, str):
            args = json.loads(args)
            print(f"[step {step}] -> {name}({args})",
file=sys.stderr)
        try:
            result = await session.call_tool(name, args)
            content = _tool_result_text(result)
        except Exception as e:
            content = json.dumps({"error": str(e)})
            messages.append({"role": "tool", "tool_name": name, "content": content})

    return "(stopped: reached max_steps without a final report)"

def main():
    ap = argparse.ArgumentParser(description="CodeQL security agent (Ollama + MCP)")
    ap.add_argument("repo_path", help="Path to the repository to analyze")
    ap.add_argument("--model", default=config.AGENT_MODEL)
    ap.add_argument("--report", default="security_report.md")
    ap.add_argument("--max-steps", type=int, default=30)
    args = ap.parse_args()

    if not Path(args.repo_path).is_dir():
        sys.exit(f"Not a directory: {args.repo_path}")

    report = asyncio.run(run_agent(args.repo_path, args.model, args.report, args.max_steps))
    print("\n" + "=" * 70)
    print(report)
    print("=" * 70)

```

```
print(f"\nReport saved to: {args.report}")

if __name__ == "__main__":
    main()
```

6. Uso

```
cd C:\Users\ladyc\Documents\TRAINEESHIP\CodeQL_Security_Agent
python agent.py "C:\path\alla\repo" --report security_report.
md
# opzioni: --model qwen2.5-coder:7b --max-steps 30
```

Il report Markdown viene salvato nel file indicato. Database CodeQL e SARIF intermedi finiscono in `_work/`.

7. Check eseguiti durante il setup

#	Check	Comando	Esito
1	Modelli Ollama disponibili	<code>ollama list</code>	OK — <code>qwen2.5-coder:14b/7b/3b</code> , <code>qwen3:8b</code> , ecc.
2	CodeQL installato	<code>codeql --version</code>	OK — <code>2.25.2</code>
3	Versione Python	<code>python --version</code>	OK — <code>3.13.13</code>
4	Pacchetto <code>mcp</code>	<code>python -c "import mcp"</code>	OK (già presente)
5	Pacchetto <code>ollama</code>	<code>pip install "ollama>=0.4.0"</code>	Installato
6	Query pack presenti	<code>codeql resolve packs</code>	Solo <code>legacy-upgrades</code> → si usa il checkout locale
7	Suite risolvibile dal checkout	<code>codeql resolve queries <suite> --search-path=...\ql</code>	OK — risolve <code>codeql/python-queries</code> • <code>suite-helpers</code>

#	Check	Comando	Esito
8	Import server	<code>python -c "import codeql_mcp_server"</code>	OK
9	Import agente	<code>python -c "import agent"</code>	OK

8. Note e decisioni

- **Niente download di pack CodeQL:** le query vengono dal checkout locale `vscode-codeql-starter/ql` via `--search-path`.
- CodeQL analizza l'intera repo come un unico database; il report viene comunque organizzato **per file** dal modello.
- Per **query custom** (es. file `*Broad.ql`) il modello può chiamare `run_custom_query` col path del `.ql`.
- Lingua di default: **Python**. Per altri linguaggi presenti nel checkout `ql`, cambiare `language=` in `create_codeql_database`.
- Variabili override: `AGENT_MODEL`, `OLLAMA_HOST`, `CODEQL_BIN`, `CODEQL_SEARCH_PATH`, `CODEQL_SUITE`, `CODEQL_TIMEOUT`.